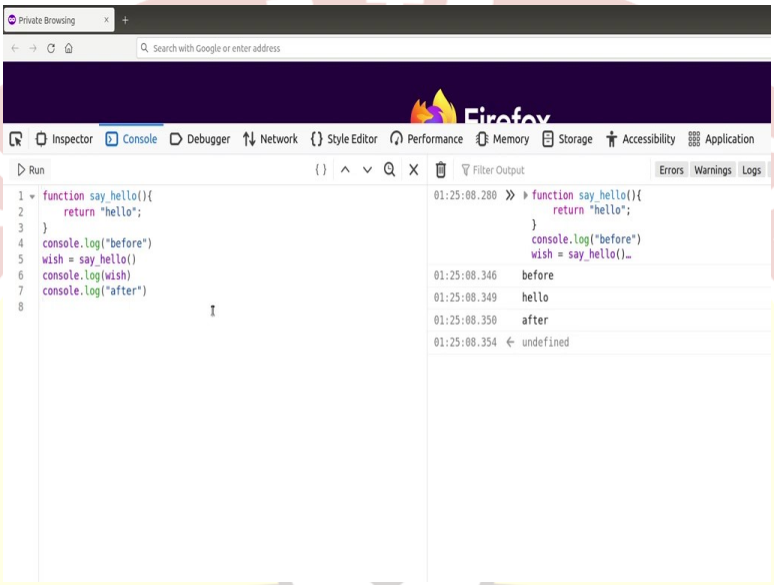
IIT Madras

ONLINE DEGREE

Modern Application Development - II
Professor Thejesh G.N
Software Consultant
Indian Institute of Technology, Madras
JavaScript – Async and Await

(Refer Slide Time: 00:14)



Welcome to the Modern Application Development II screencast. In this screencast, we will explore JavaScript Async functions and Await keyword. For this, you will need a browser Firefox or Chrome. Here, I have Firefox and I have opened a developer console. You can just go to developer tools, more and click on these developer tools, and it should open and go to the console mode.

In the most basic form, JavaScript is synchronous, which means it will execute from top to bottom line by line in a blocking way, in a single thread, in which only one operation at any point runs. But that said, in web browser, we have certain function and APIs that allows us to register functions or they are round in a synchronous mode. For example, there is a set timeout, which you can run in a synchronous way. Almost all user interactions which are with respect to mouse or keyboard are asynchronous.

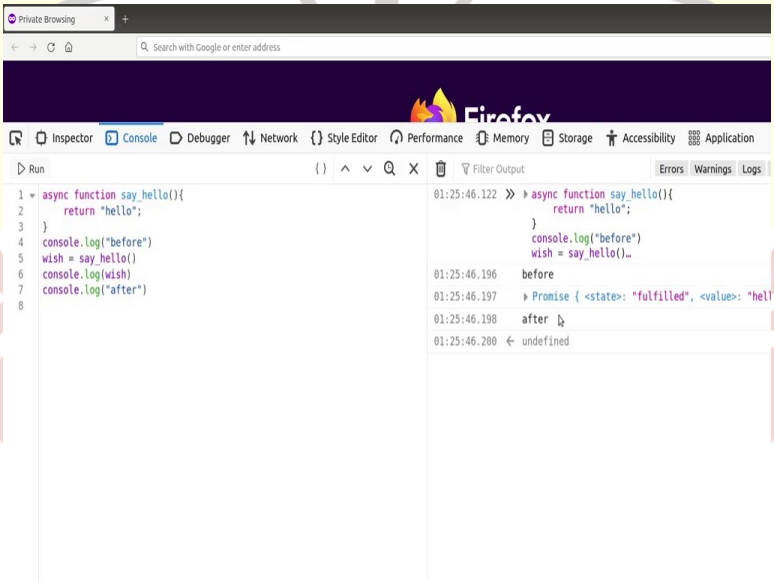
They run on on-click on mouse over any of certain interaction. Their engine does not wait for you. It only waits for the event. When the event happens, it just runs asynchronously. It does not wait the running code until then. And then arrival of data or downloading of data or accessing

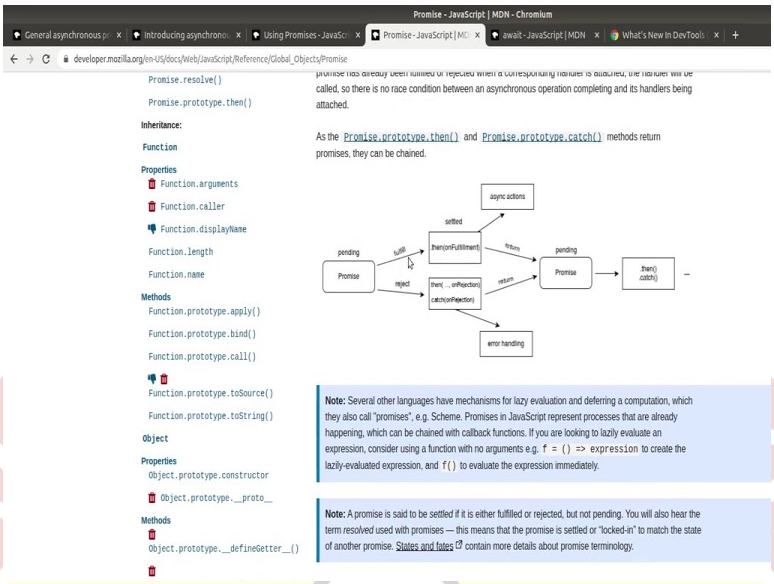
some data we have already heard the word ajax, which is asynchronous JavaScript and XML, which is by definition, asynchronous, which means when you request for the data, it happens a synchronously. It does not stop the main loop.

This means your code can do several things. For example, if it is doing some network activity, it can also do something locally, kind of most efficiently using and can speed up the things. In this session, we will learn how to make async functions and also how to use a keyword called await. Let us start with a function called hello, which looks like this. This is just a function called say hello, it just returns a string called hello. Now, let us call it, can just say wish is equal to say hello and you can just run it. Just before running it, I will just had some comment before and after. So, we know it is running and also we will print.

So, let us see all of it. I will just enable the timing, so it prints the time of running shows timestamps. Now, I can just run this. Now, we can see it is running synchronously, because before was called, hello was called, then after was called. Hello comes from here, now logging the wish. So, they are all running synchronously one by one, top to bottom. You can see that.

(Refer Slide Time: 03:50)



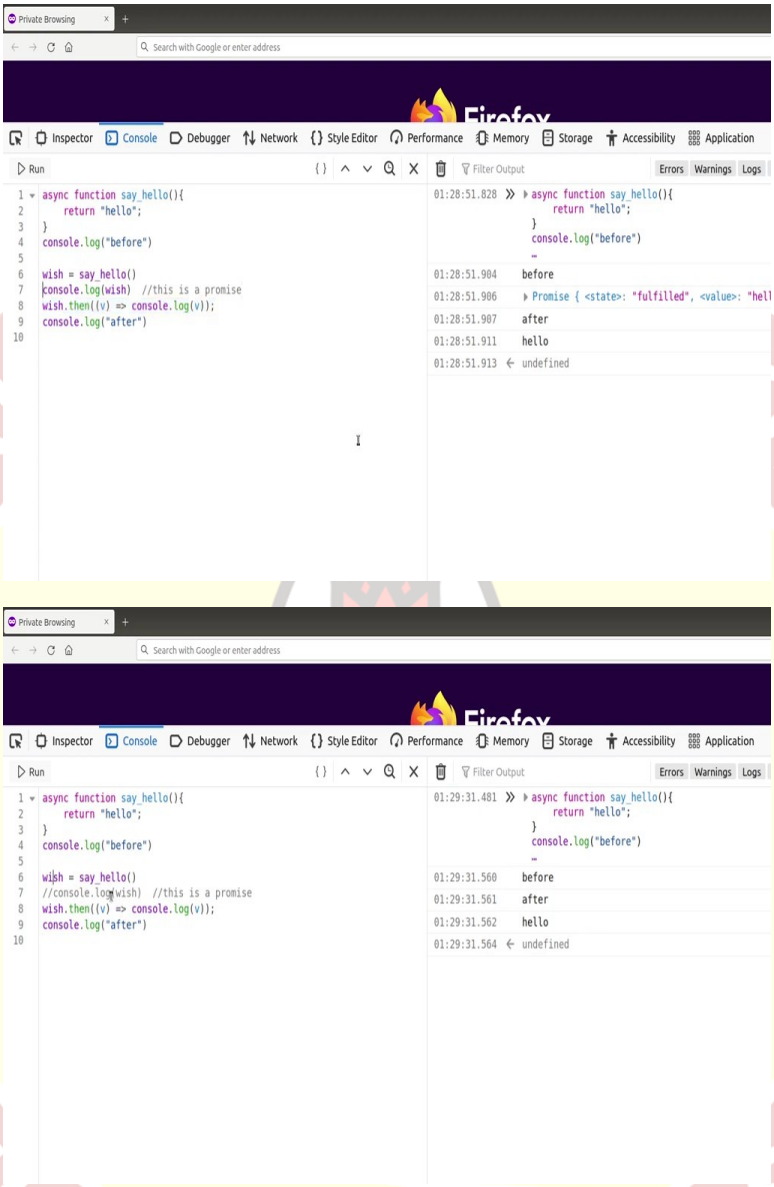


Now, we can easily convert this into async. That can be done by just adding a keyword called async in front of the function. Now, we can run this. Here, you can already see a difference. It does not actually return the response, hello. It returns something called a promise and then it continues.

Now, promise is a key, is a way of browser telling you that I am going to return the actual value sometime later. You can see that here. If you go to MDN documents, you can write more about it, but promise is an object which can either be fulfilled at the later stage or can be rejected at the later stage. When I actually get it, it will be still in the pending stage. Here we can see that it is already been fulfilled and the value is hello. But we will see.

So, it can either be fulfilled or rejected. If it fulfilled, we will get actually some value which we can do. If it is rejected, we will either get rejection or we can get an error. You can read more about promises on the Mozilla developer web docs. I would not go there further. But this is what you get.

(Refer Slide Time: 05:25)



Now, once you get a promise, you have to actually get data after the promise. You cannot just use wish directly. So, what do we do generally, is something called thenable or use then keyword. We will do let us say message is equal to wish. Let us not do that. Let us do directly wish.then. This will resolve the promise, which means it will wait till the promise changes the state from pending to fulfilled and then it runs something.

What does it run? It returns a value and then we can call a function on that value. This is an arrow function or anonymous function defined in a short form. And I will just call console.log, I am printing the v. V here is just this whatever the promises returning us value. It can, we can call

anything. I can call message or anything, but I am just calling v here. And then we are printing that. Now, let us see how it goes.

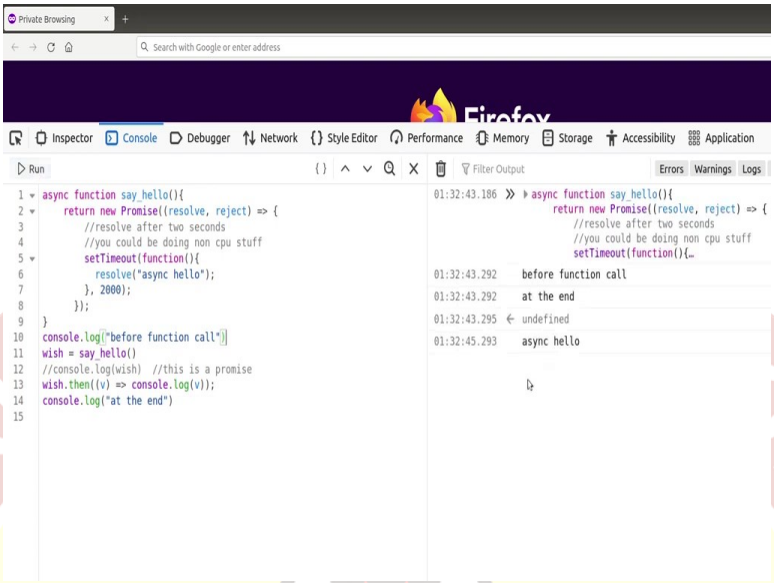
Now, what are we doing? First, we have a function defined, then we calling before, then we are actually calling the function and printing the return, which is actually a promise. This is a promise. Then on success we are actually extracting the value out of promise and trying to print it. We can see that it is in line number eight and then we are printing after in line number nine. Let us run this.

Now, you see a difference here. Even the promise got printed first. But after promise we got this after printed. Hello came much later. Hello is actually printed by this line, but it came later. That is because it took some time to execute and it returned asynchronously. That is why the thing got exchanged. If you, if I actually mask this line, then you can see that before, after and hello. So, the response got later and then after printing after. So, here is where you can see that it is not running synchronously. It is running asynchronously. It is not going, it is not like we got called first and then we printed the hello and then we printed after.

We just got called, which means it went into us separate thing to run by itself, but the main loop continued with console dot log to print after. Then when the response was succeeded, then the promise was resolved and then you got the printed response called hello. This is how a synchronous JavaScript works. And this is how we usually use promise that you use keyword called or a function called then which you used to resolve and then can have both fulfilled or success or failed. We call it either resolved or rejected.

Now, generally, you do not do just directly do return, you will have a function which will do many things. So, you usually use another keyword called resolve. And this flow also becomes more visible when you have a delay here. Like for example, if you are doing some work here, which is not CPU bound, let us say it was, you are trying to get some data from outside or you are trying to read database which is network call, things like that, then it becomes apparent that how asynchronous can be useful.

(Refer Slide Time: 09:34)



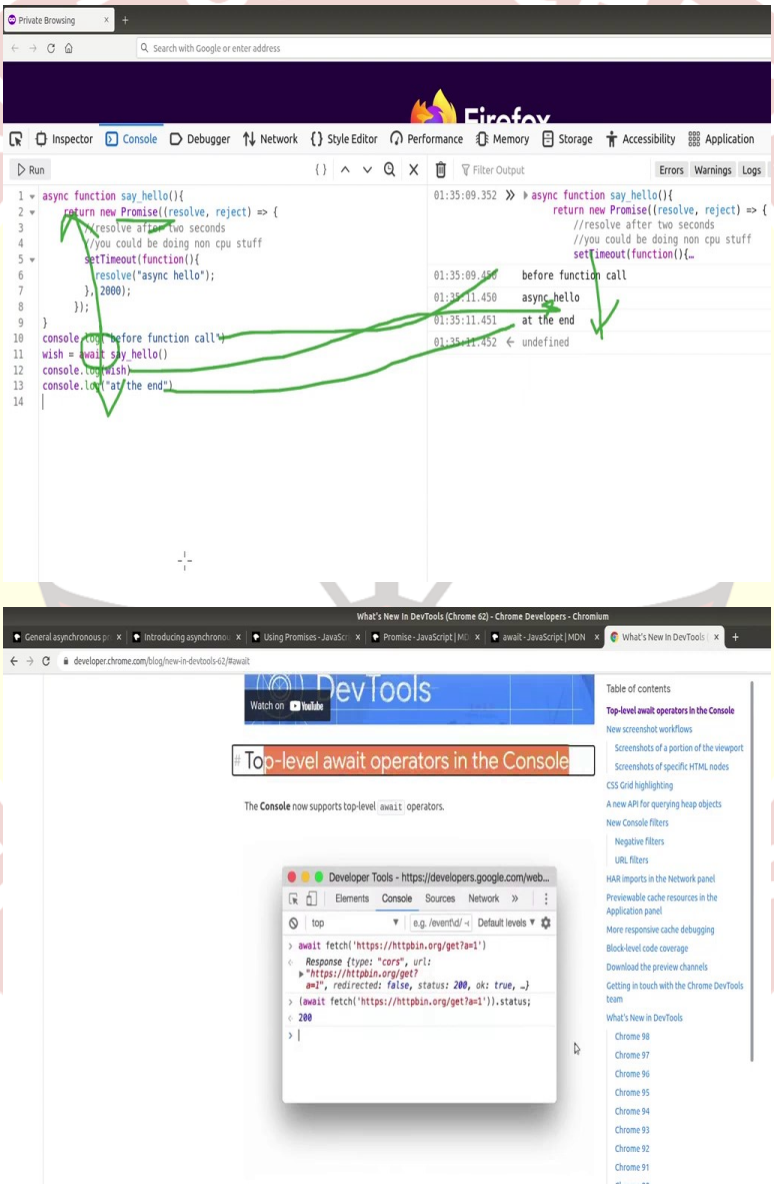
Now, let us go ahead and do some changes to this async function to make it standard way by using function called resolve instead of actually directly returning, which is also actually turns out to be promised, but this is more what we would write in a function if you are going to write a function. So, here what I am doing, and just to give a sense of how this goes, I am adding a delay of 2,000 second and then resolving saying async hello. So, let us just had before function call at the end.

So, I have an async hello in between. It gets called after 2 seconds. 2,000 is 2 seconds, set timeout, keeps the thing aside and calls after certain timeout, like, for example, here 2 second calls this function, which is async hello, which results this whole promise and then we can print in the main loop.

Let us clear this, let us say, let us run this and see what happens. Now, if I run this here, you can clearly see that a printed before the function call as usual, and then we called say hello, and then we printed the response, fulfilled response, and then we printed at the end. Now, we can see that at the end got printed even before async hello. Async hello came 2 seconds later. For example, 43 here, 45 here, which means our main loop did not get blocked here or rather here. It continued with the execution further, but when the response came or when the promise got resolved, it printed the result. So, this makes it efficient use of CPU or time as well as fast. This is asynchronous program. This should make it clear.

Then there are places where you want to convert or where you want to run these things in sync. Let us say you are trying to get some data from a web service. And based on that, only then you want to continue with the rest of the functionalities. You do not want to do anything until you get the data. And in that case you want to wait until it gets resolved. For that we use a word called await.

(Refer Slide Time: 12:30)

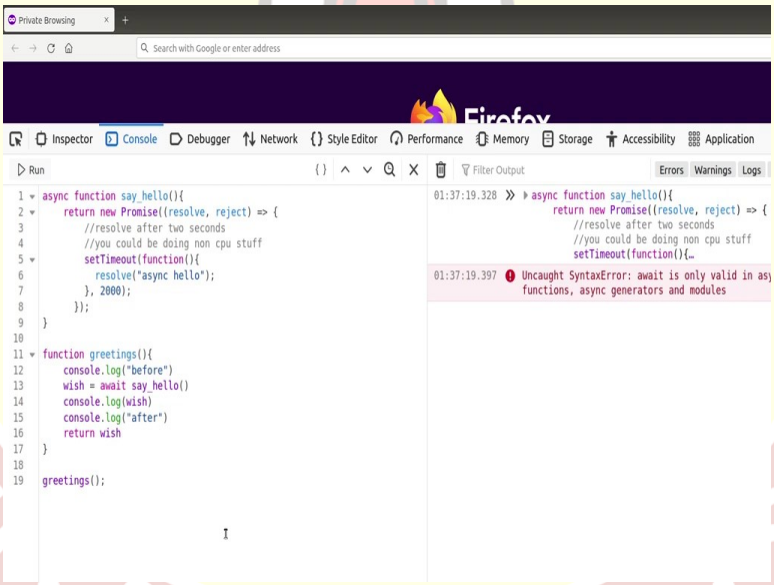


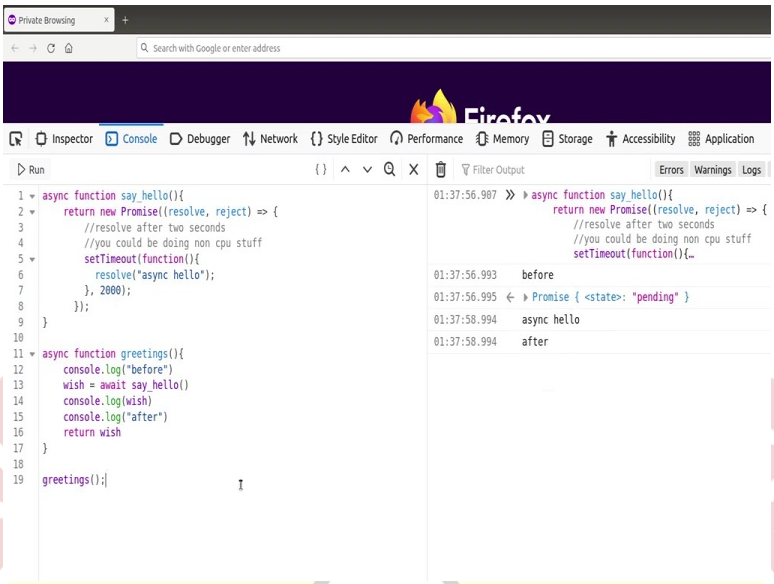
So, that is done just by adding to the function, whatever the async function we are calling, just add await in front of it. So, now, let us run. So, you had this before, like reversed. Now, let us run this. Let us clear this and run this. Now, if you see, we are not printed here, let us print it. Let

us print the wish. I will just clear this. Let us run this. Now, you can see that it synchronous. This is this and this is this and this is this. They are all running in synchronous mode like this. The way it is here, the same way it is here. That is because of the keyword await. This makes call of this and actually waits till this promise is result. Whatever the promise it is returning, it waits till the promises resolved, only then it continues. Hence, it becomes almost like a synchronous thing.

Now, there is a catch here. You cannot call await in just, in the main loop directly. If you want to, the await can be called only into another async function. Now, since we are experimenting inside the browser console, it is an exception given to developers to experiment. If you look at the dev tools, it is only in the developer console of both Chrome and in the Firefox. They have added an exception where await resolves in the main loop. Otherwise, it usually does not resolve, it will throw an error. I will give an example.

(Refer Slide Time: 14:50)

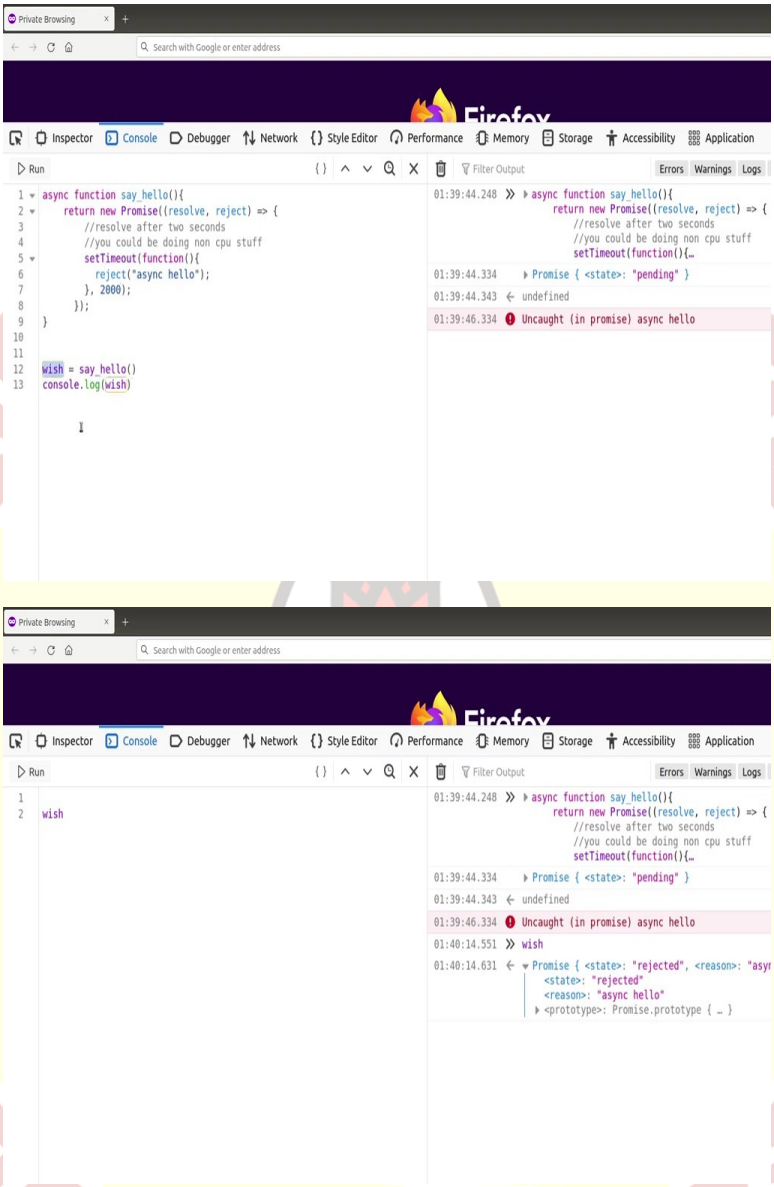




Let us say we have two functions. I am just going to clear this. The first function is the same thing, async hello. The second function is called greetings, which actually calls the first function. And then it is calling await to resolve all the promises before it continues. Now, this function is a synchronous function. We do not, we have not added async. Now, if I call greetings and if I run it, let us run it, you can see that it throws an error. Await is only valid in async function, async generators and modules. So, what it is saying is, this await can be used only inside async functions or async modules or async generators. So, to make this work, we have to add async here. Then this will return a result.

Let us try and run. So, it is a promise, it is resolving, then it got async hello and after. So, now you know that you got the result. It is calling before then it called wish await say hello and then it is waiting for it to resolve. This is waiting for it to resolve. It got resolved, then it printed say, async hello, then returned after then return the whole function back to this. So, this is how you call an await. If you want to call an await, it has to be inside another async function. You cannot directly call it. If you directly call it, it will throw a syntax error.

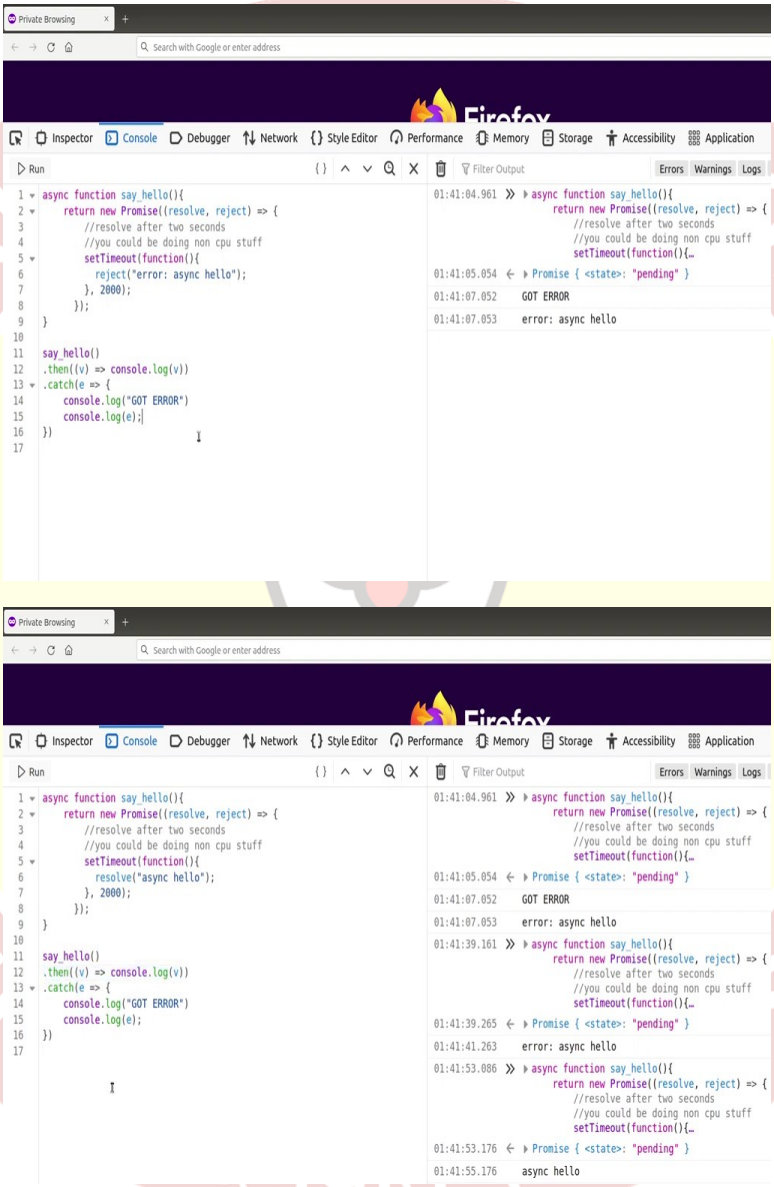
(Refer Slide Time: 17:15)

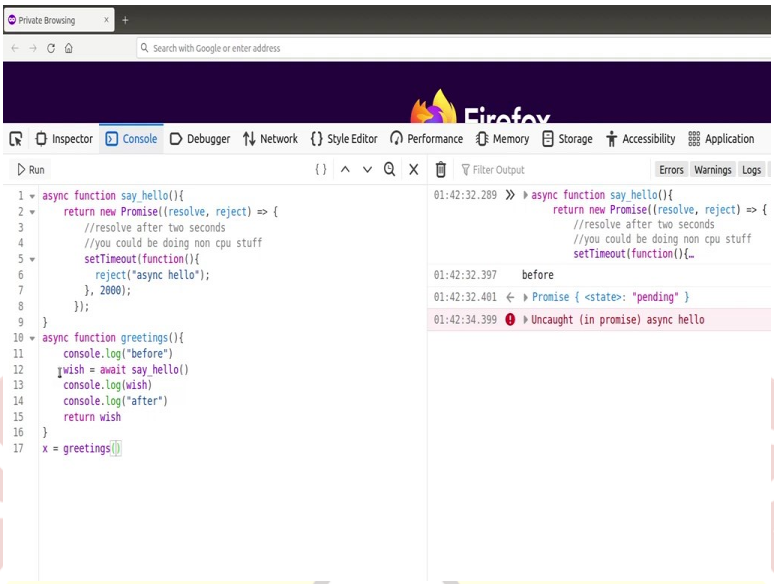


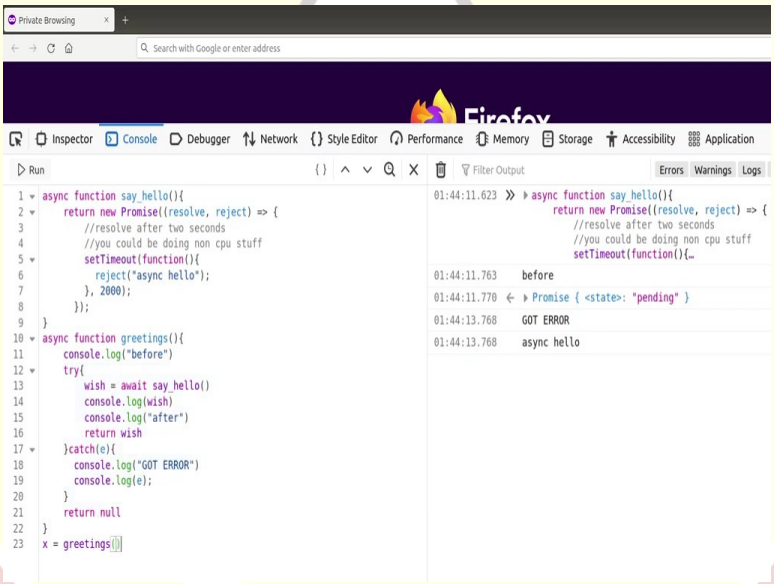
Let us do, so what happens if there is a, some kind of an error. I mean, let us say instead of resolving, somebody rejected it. That is what happens generally. Let us hide this for a while and call say hello directly. Wish is equal to say hello and we will just print console log the wish. Let us run this. I am just going to clear this. It is in pending. You can see uncaught in promise async hello. So, we are not catching, but we know that it is thrown an error. Now, we can just check what is the value of wish just to see. I am just going to print, remove this and print just the value of wish again. You can see that it is rejected. And the reason is async hello. It is been rejected.

So, when you actually have asynchronous way of coding, you should also handle errors. To handle errors what we do, we will actually catch an error just the way we caught the fulfilled thing we also catch an error.

(Refer Slide Time: 18:57)





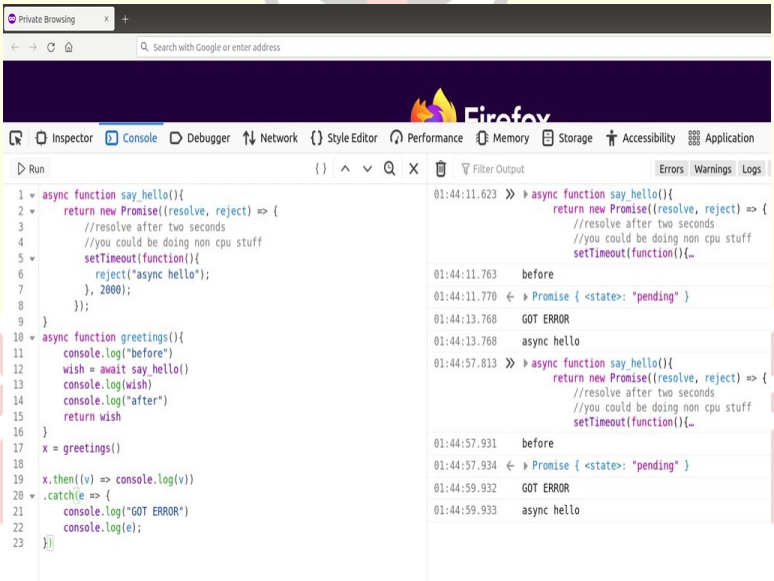


So, this is how it would look for example. It is a same function just like that we did dot then. When it succeeds it returns, it goes here. If there is an error or if there is a rejection, it goes here and prints there. We can run this and see. Here, so it got the promise, it waited for it to resolve for 2 seconds, then it would have gone here if it was resolved, but this is reject, hence it went here and said printed, got error and then printed the error, which is actually the message that comes as part of reject. I mean, if it was resolved, it would not have printed this. It could have just printed, sorry. See it did not print got error. It did not print. So, we can try again just to check. So, it printed just async hello, it did not go here. Just went here directly. That is how you catch an error.

Now, what happens? How do you catch an error if we are actually doing await? We can try that again. Let us go back to our greetings. Now, let us say this is what we are doing. We are doing everything correct here. But here it was a reject. Clear this, let us run this. So, here is again saying it is an error. It is throwing an error on line 16, the end of this. You can check in the DOM console where exactly it is throwing an error the end of this function.

So, what we will do is we can actually fits an await. We can do standard try catch, can do a standard try, you can put this inside here, I mean, you can put all of them here. Just print error, and then console dot. This is like a standard way of catching errors, like how would you do it in a synchronous programming, same way you will catch an error here in await condition. I will just return null. I will return a null here. Let us clear this and run this. And see now it should actually print caught error and also log the error. Let us run, wait for 2 seconds into got error and logged error. So, if you are actually using synchronous, you will use try catch like this. If you are actually using asynchronous way or if you are using without await, then you can do dot then.

(Refer Slide Time: 22:47)



It can also continue further, like for example, if let us say you did not do this here. I mean, you have x here, x dot then, you can do the same thing here. I think this will also work. It is a same thing as we did like before. You can, I am just showing you the difference between how we would do if it, we are trying to catch error in an await function, awaited function, you just

surround it by try catch. If it is a promise based one, then you actually catch using dot catch. That is how you do error handling.

So, basically, what we saw is we can easily convert any function into async function. And then execute it asynchronously. Some of the functions, built-in functions which we will see next, like fetch, are asynchronous, but they can be made into synchronous using a keyword called await. We also saw how to catch errors in case of asynchronous functions, and in case of awaited functions, how to catch an error using try catch. Whether we want to run some code asynchronously or synchronously depends exactly on what we are trying to achieve. Over time we will understand and plan way to use asynchronous functions and way to use synchronous functions. Thank you so much for watching. Bye, bye.